700105 Simulation & Concurrency

UNIVERSITY of HULL

Mid-Module Review

- Go to menti.com

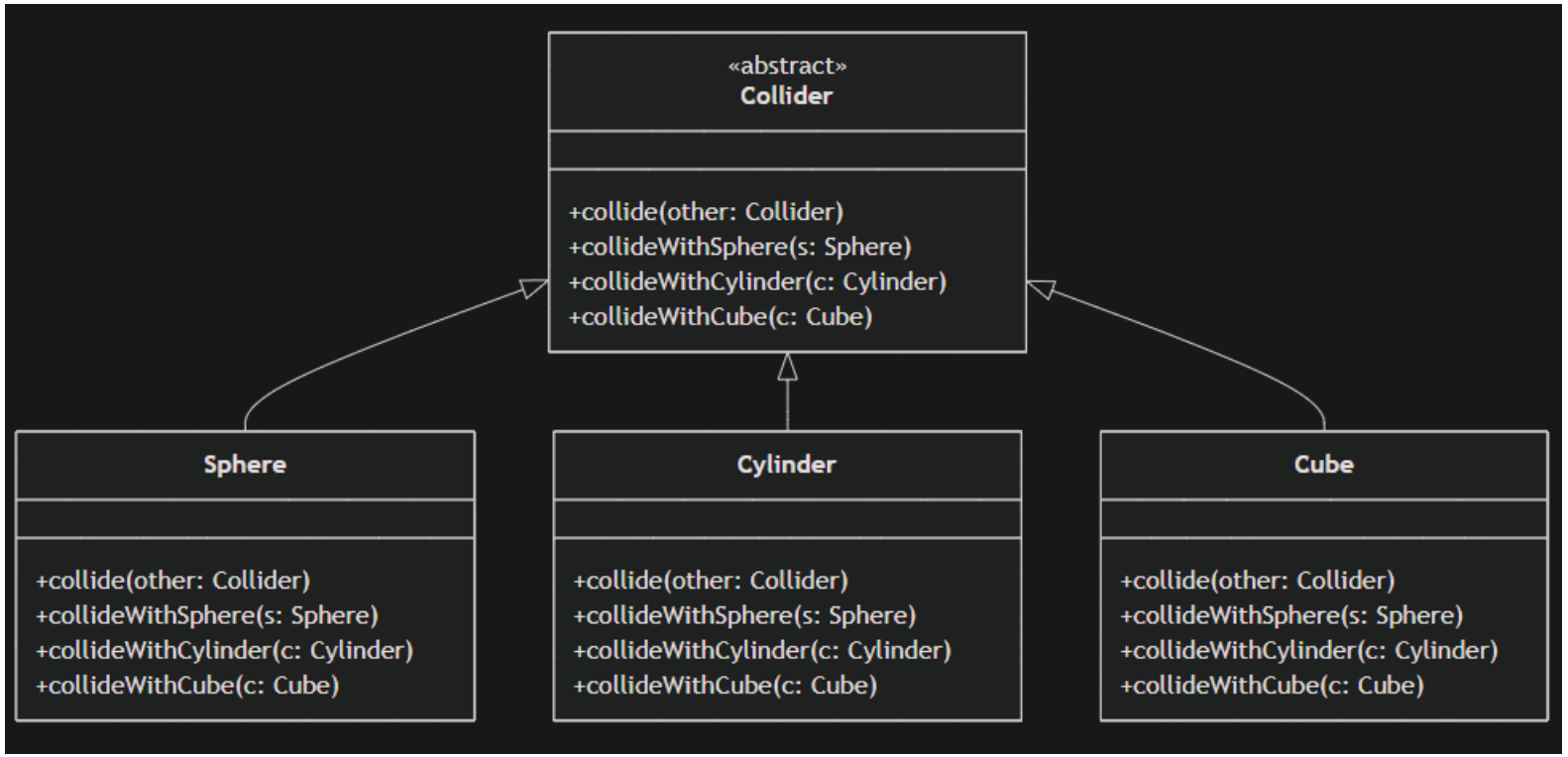
Double Dispatch

Double Dispatch

- A technique where a virtual method call is resolved at *runtime* based on the type of *two objects* instead of just one

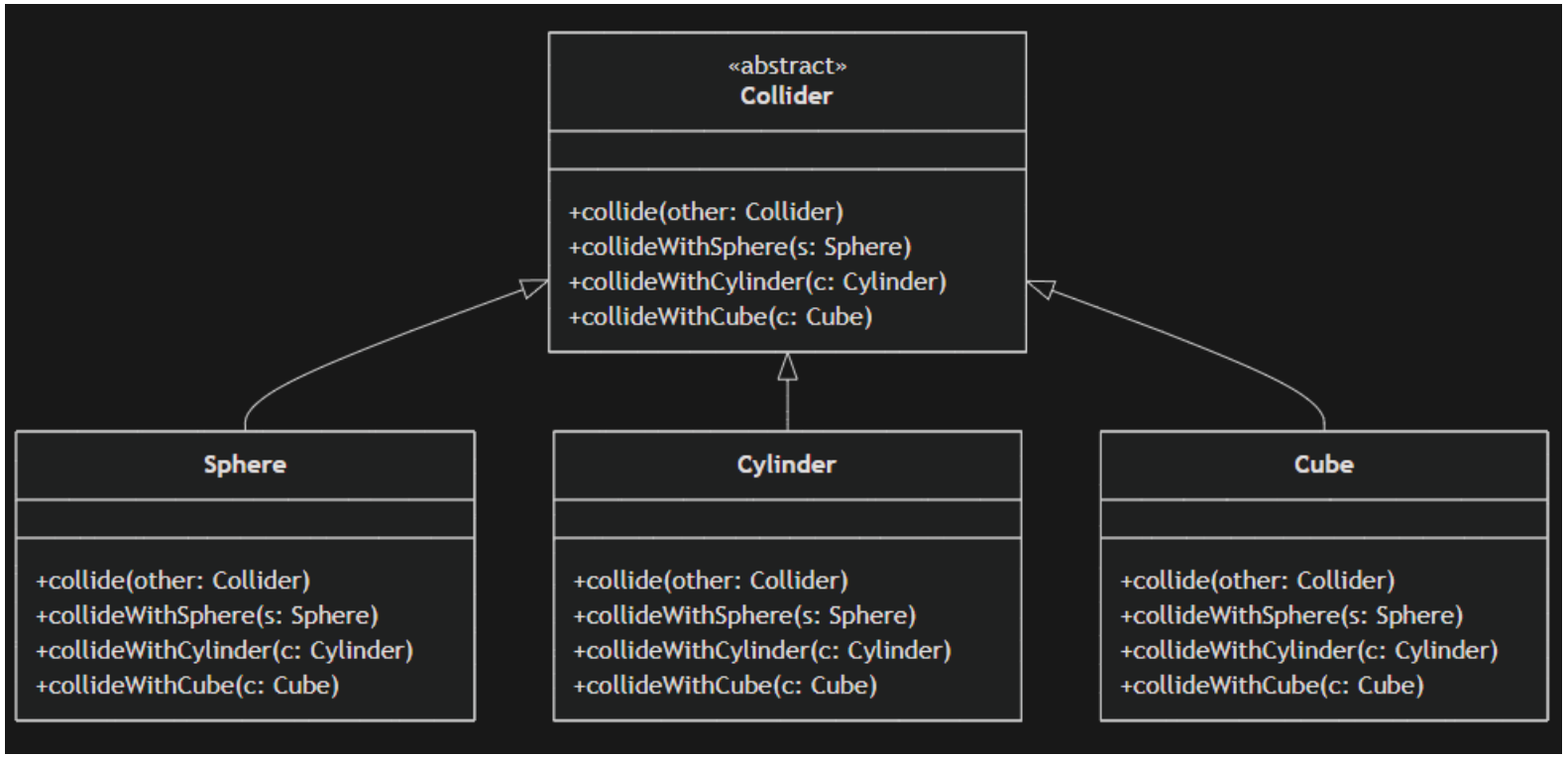
Colliders and Shapes

- Suppose at run time we don't know the type of two colliders c1 and c2
- Call c1.collide(c2);
- This first dispatch reveals that c1 is a Sphere (for example) and calls the specific collide method in the Sphere class – passing in c2



Colliders and Shapes

- The collide method in Sphere calls collideWithSphere on the other collider, passing itself
- c2.collideWithSphere(this);
- This second dispatch reveals that c2 is a Cylinder (for example) and calls the specific collideWithSphere method in the Cylinder class – passing in c1



Where should the actual collision code be?

- Conceptually the code doesn’t really belong to one shape or the other
- To keep it clean and avoid repetition arguably it’s best to put that code in some static method that “belongs” to neither shape

The story so far

The story so far

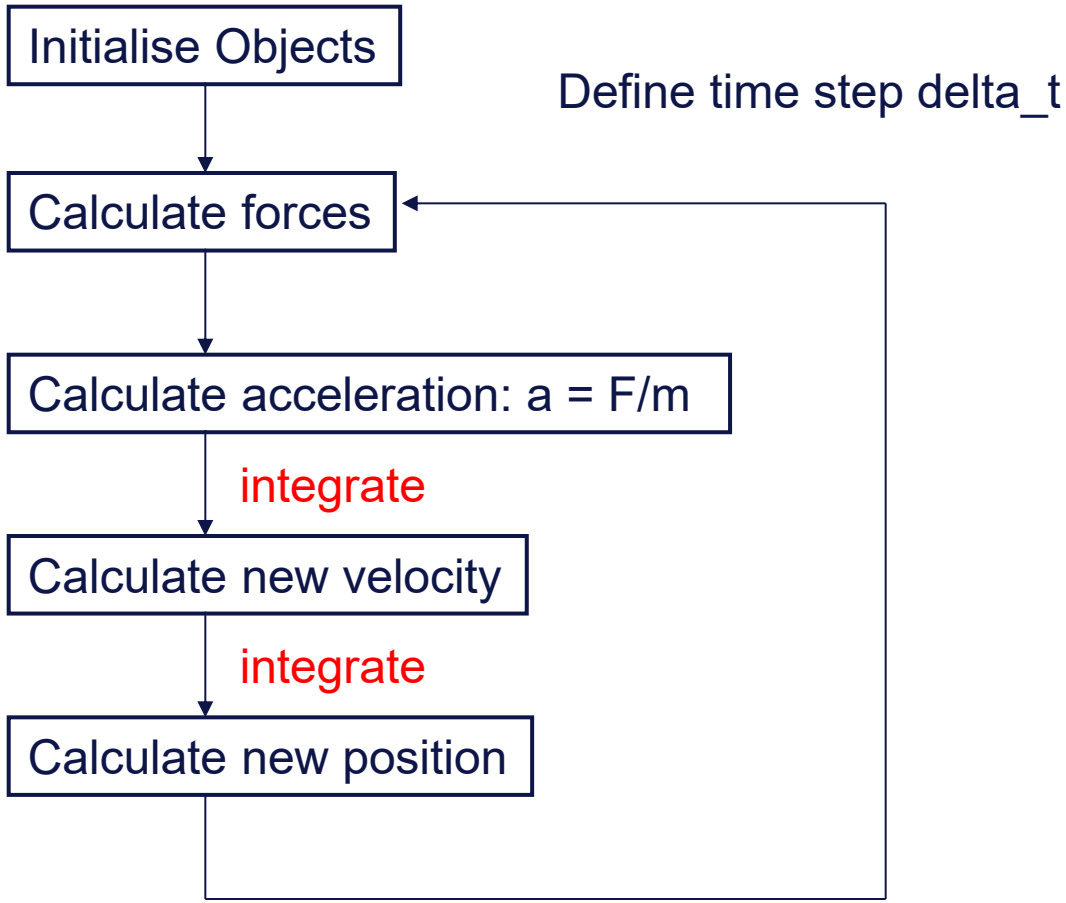
- Updating State
  - Calculating linear forces and torques
    - Gravity
  - Calculating new velocities and positions (particle/projectile motion and rigid bodies)
    - Integration methods (Euler, Semi-Implicit Euler, Verlet, Runge-Kutta)
- Collisions - Geometric Objects (Shapes)
  - Primitives (point, line, line segment, plane, sphere, cylinder, capsule, cuboid, triangle, convex hull)
  - Methods (Inside-Outside, Intersects, Contains, SAT, GJK)
- Collision Response
  - Calculating new velocities or forces after collision using impulses or forces
  - Conservation of linear and angular momentum (including inertia)

Rotating things

- Last week we looked at orientation, angular displacement and angular velocity
- By considering nested spaces we can treat linear motion of the centre of mass and orientation around the centre of mass separately

Springy Things

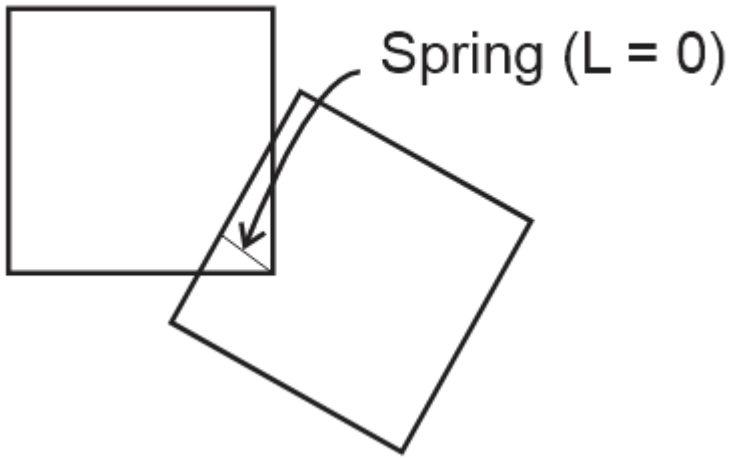
1-dimensional simulation

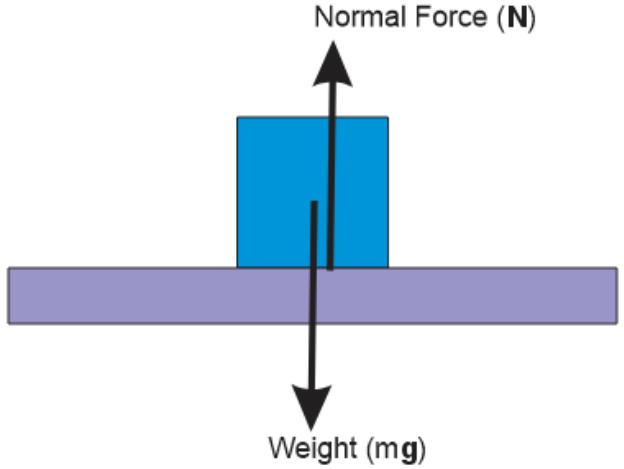


Note: No collision detection/response included at the moment

Springs

- Some approaches allow a little penetration between objects, and calculate a force to push them apart again





Hooke's Law

- $F=kx$

Where:

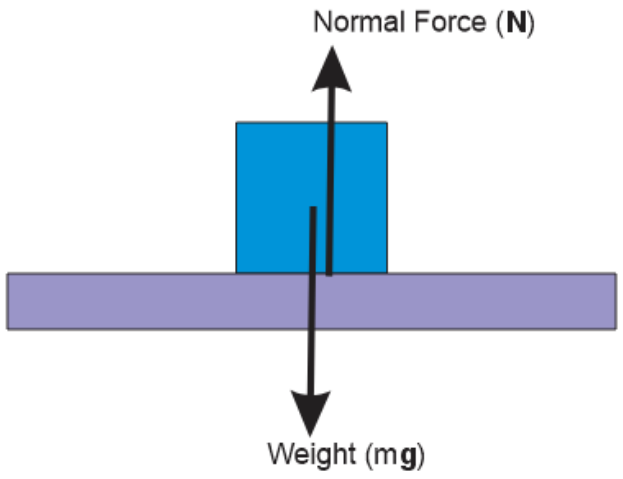
- **F** is the force applied (in Newtons, N)
- **k** is the spring constant (measuring stiffness, in N/m)
- **x** is the displacement or stretch from the equilibrium position (in meters, m)
  - Note, this describes the ***magnitude*** of the force

Penetration Correction

- **Collision Response and Penetration Correction:**
- When two rigid bodies collide and slightly overlap (penetrate), the physics engine applies a **spring-like force** to push them apart.
- The force is calculated using a modified Hooke's Law:
- $$F = -kx$$
- **x** represents how much the objects overlap (penetration depth).
- **k** is a stiffness constant controlling how quickly they separate.

Stabilizing Rigid Bodies

- Soft spring forces are used to **correct small overlaps** that occur due to floating-point inaccuracies, keeping objects from sinking into one another.



Damping

- Often, a damping term is added to simulate friction, which reduces oscillations over time:
- $F = -kx - bv$
  - **b**: Damping coefficient (controls how fast the motion slows)
  - **v**: Velocity of the moving object

Soft Objects

Deformable Objects

- Things that change shape due to external forces
- Examples
  - 1-dimensional
    - Rope
  - 2-dimensional
    - Cloth, paper, skin
  - 3-dimensional
    - Organs, jelly, football

Approach – Spring and Dashpot

- Model a rope, cloth or volume as a series of small masses
- Apply forces using springs and dashpots (dampeners)
- Reasonably simple
- Based on modelling objects as a collection of point masses connected by springs
- Uses Hook's Law for springs
- Not a 'true' physical representation
- Prone to energy explosions
- Works best for 1- and 2-dimensional objects
- Doesn't give realistic forces and hence not really suitable for haptic feedback systems
- Very popular …

Approach – Finite Elements Method

- Divide a large problem into lots of “finite elements”
- Complex
- Based on sound engineering theory
- An accurate mathematical representation of reality
- Works best for 2- and 3-dimensional objects
- Can give accurate forces for haptic feedback
- Less popular due to mathematical complexity and high processing requirements
- A fast, tuneable version developed at Hull

Approach – Position Based Dynamics

- Another approach that works directly on positions
- https://matthias-research.github.io/pages/publications/posBasedDyn.pdf

Spring and Dashpot

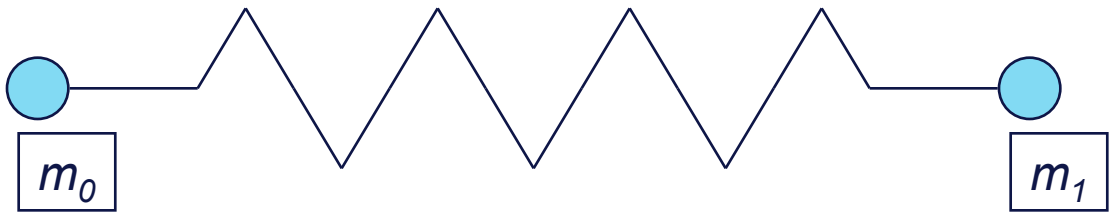
- Overview
  - Springs are used to model the elasticity of an object
  - Dashpot is used to damp down oscillations
  - Calculation for motion are based upon Newton’s Laws and Hook’s Law
  - Mathematically simple, but Euler integration can create considerable problems

Spring and Dashpot

- Concept
  - Create a mesh of the deformable object
  - Replace the polygon edges with spring/dashpot elements
  - Replace vertices with point masses
  - Gathers forces (gravity, wind, springs, dampers etc) on the masses and use numerical integration to find approximate motion

Spring and Dashpot

- Mathematical background
  - Springs have no mass
  - Total mass of the object is distributed evenly across the masses at the vertices
  - Considering simple example:
    - Let m_0 and m_1 be the masses at either end of a spring
    - Let the rest length of the spring be L_r and the spring constant be k_s
    - k_s determines the strength of the spring



Spring and Dashpot

- Define the force produced by the spring as:

$$F_s = k_s\left(L - L_r\right)$$

- L is the current length of the spring
- F_s acts on, say m_0, and $-F_s$ acts on m_1
- Later these equations will be formulated in vector notation, which helps resolve the direction of forces

Spring and Dashpot

- Define the damping force F_d as:

$$F_d = -k_d\left(v_0 - v_1\right)$$

- k_d is the damping coefficient of the spring
- v_0 and v_1 are the velocities of the masses m_0 and m_1 respectively
- Once again, if F_d acts on m_0 then $-F_d$ acts on m_1
- F_s and F_d act along the spring, although the total forces on the masses may not do so

Spring and Dashpot

- In vector notation
  - Let the position of m_i be p_i with velocity v_i
  - The total force F_i on m_i from the spring is:

$$\mathbf{L} = (\mathbf{p}_1 - \mathbf{p}_0)$$

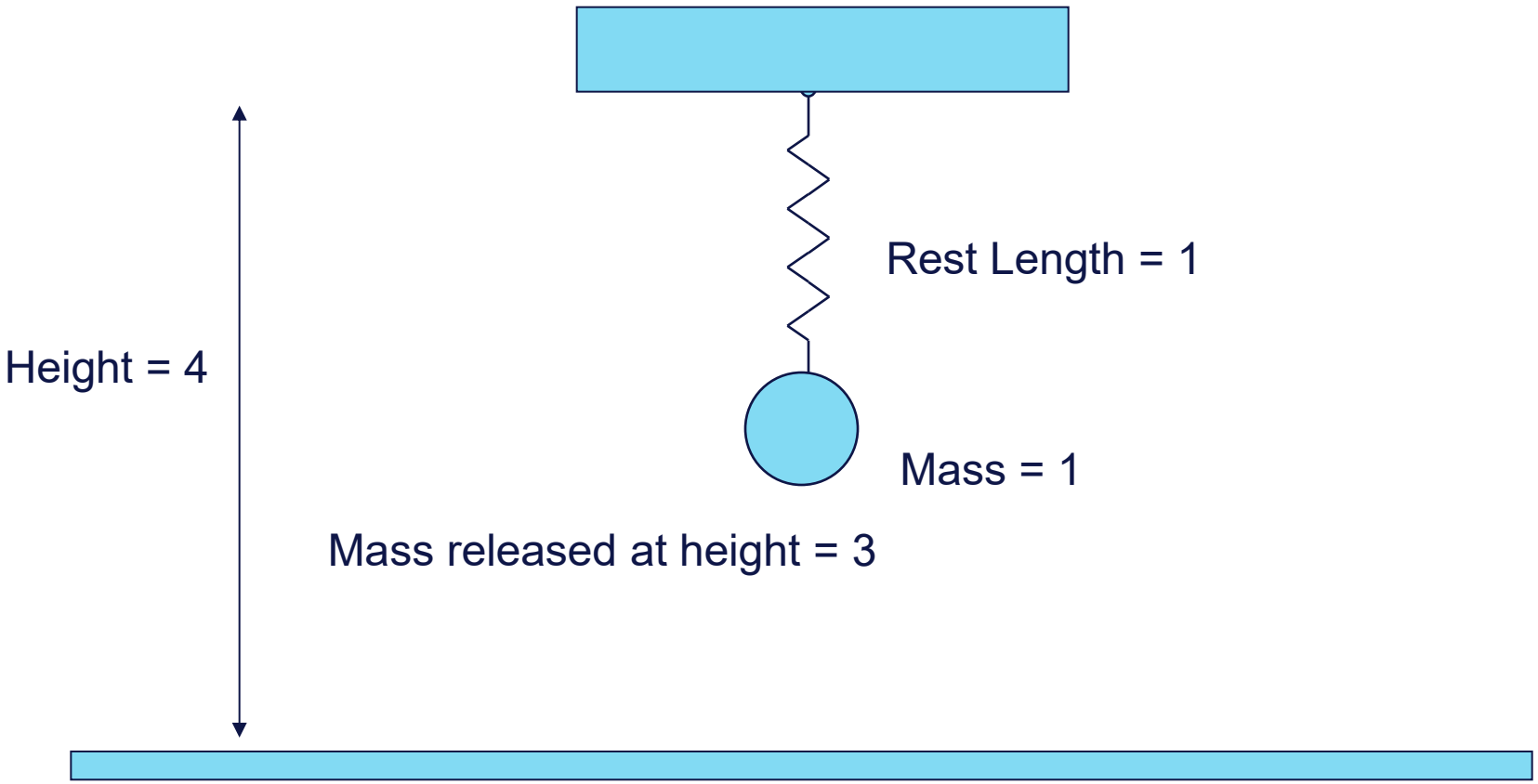
$$L = |\mathbf{L}|$$

$$\mathbf{F}_0 = \left(k_s(L - L_r) - k_d\left((\mathbf{v}_0 - \mathbf{v}_1).\mathbf{L}/L\right)\right)\mathbf{L}/L$$

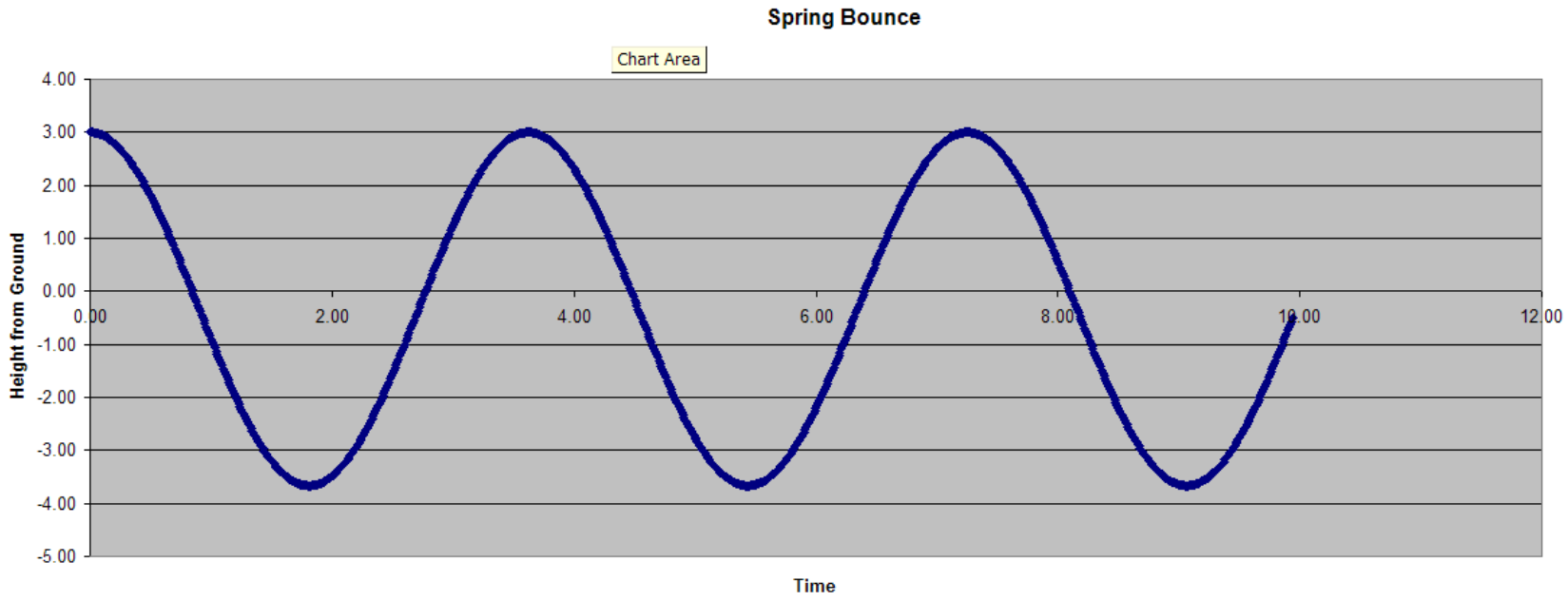
- Note: $F_1 = -F_0$

Spring and Dashpot

- Hanging spring

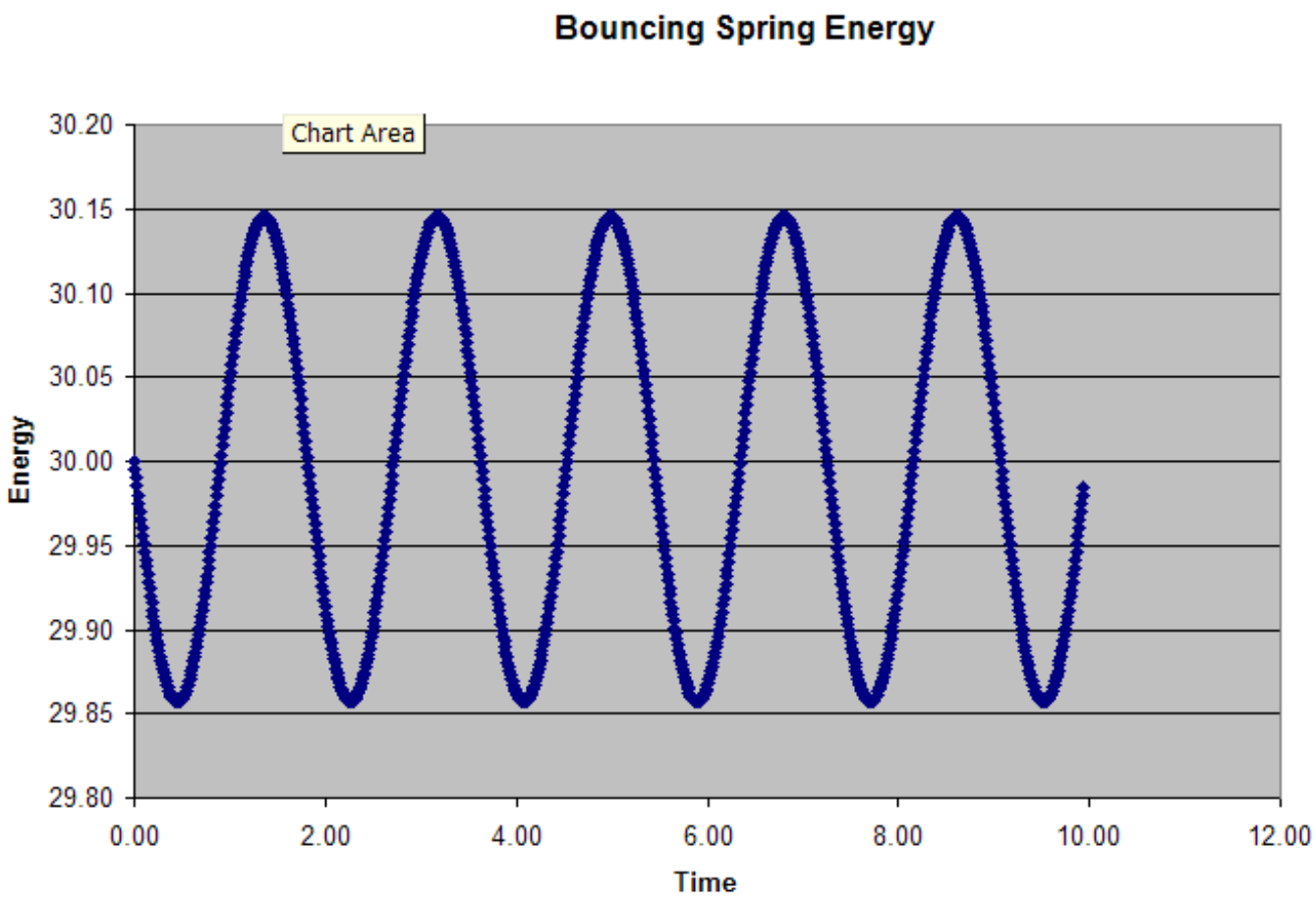


Spring and Dashpot



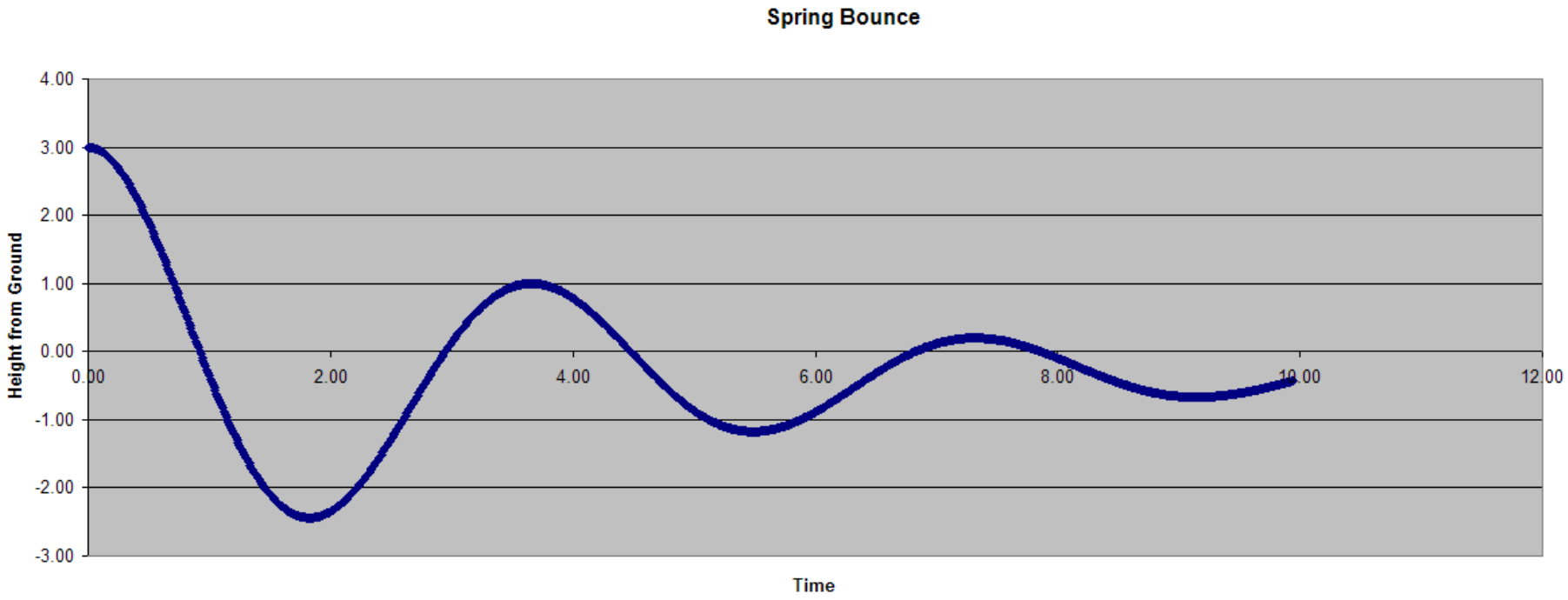
Spring coefficient = 3, damping = 0, time step = 0.01

Spring and Dashpot



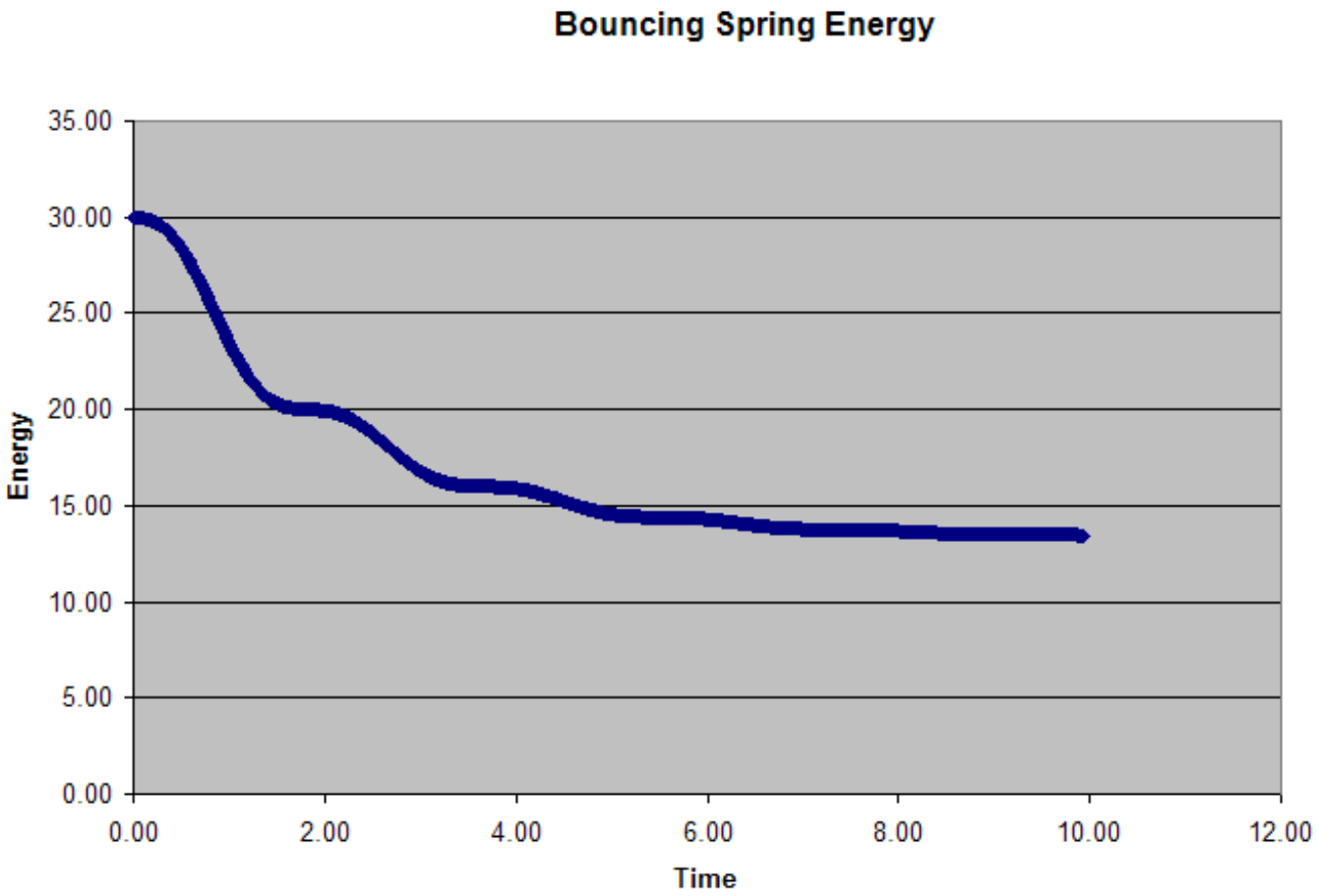
Spring coefficient = 3, damping = 0, time step = 0.01
Energy should stay constant, but slight variance

Spring and Dashpot



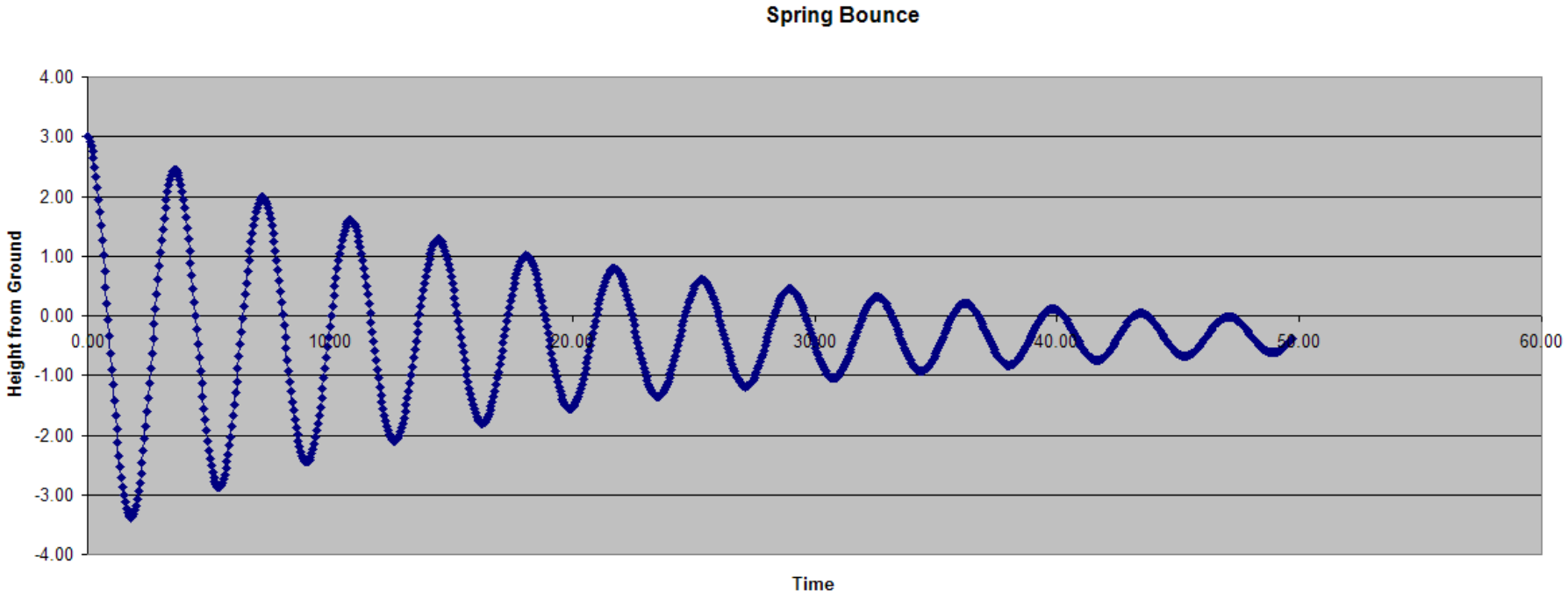
Spring coefficient = 3, damping = 0.5, time step = 0.01

Spring and Dashpot



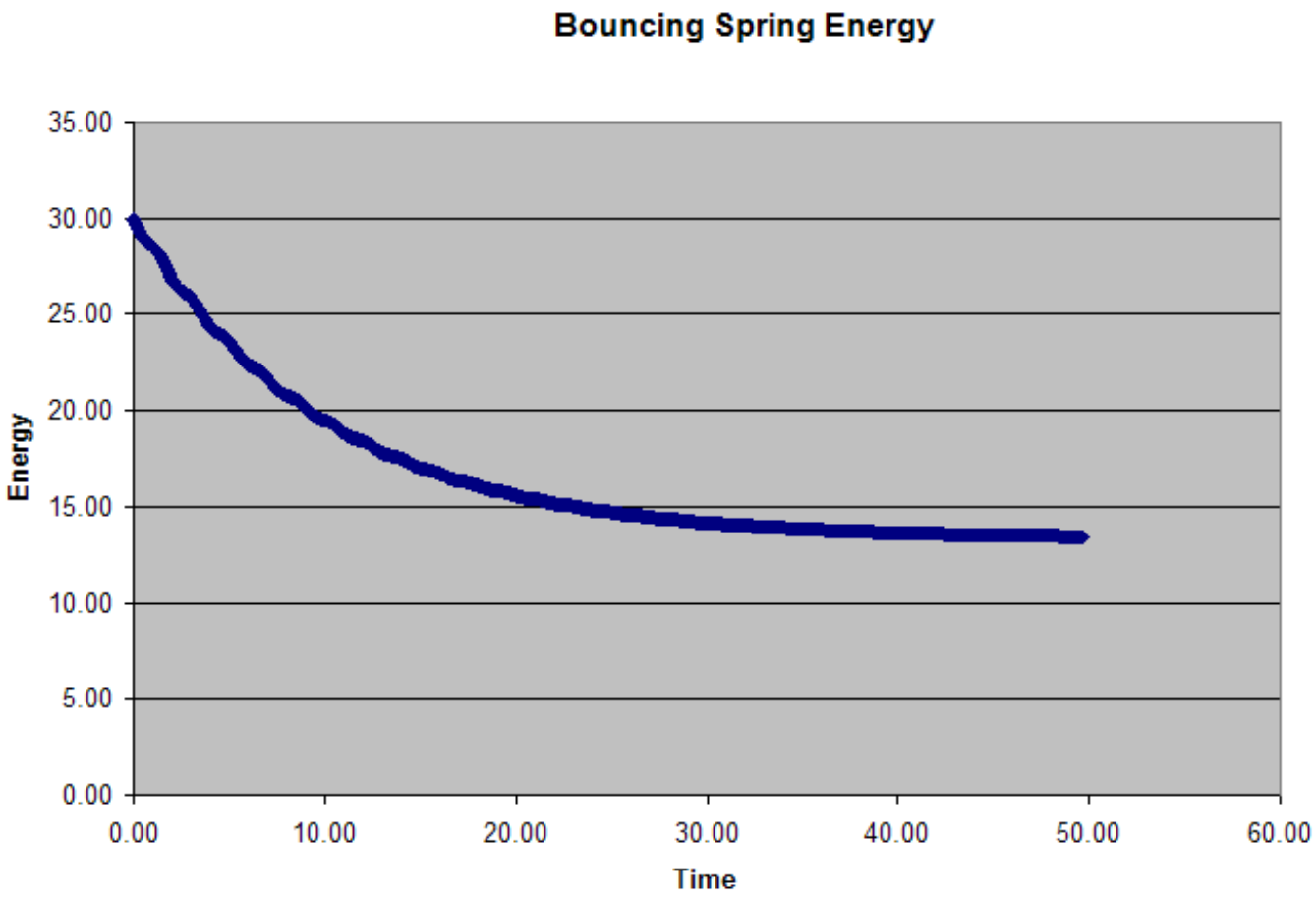
Spring coefficient = 3, damping = 0.5, time step = 0.01
Energy decrease due to damping

Spring and Dashpot



Spring coefficient = 3, damping = 0.1, time step = 0.05

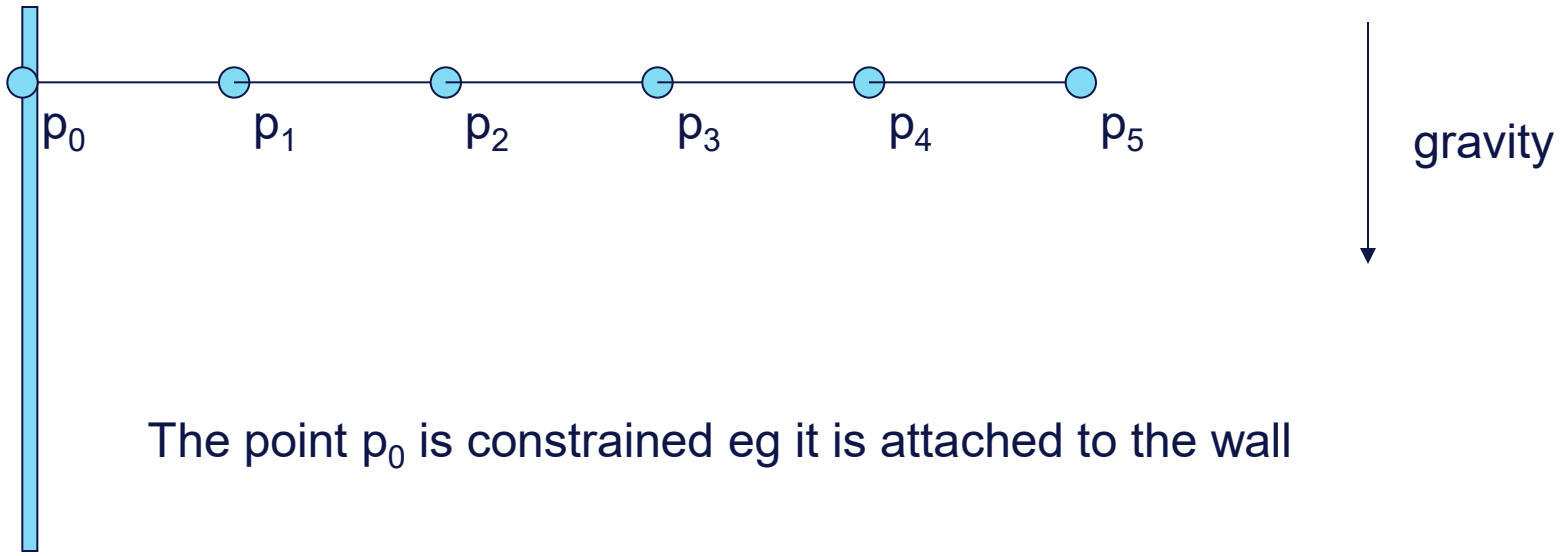
Spring and Dashpot



Spring coefficient = 3, damping = 0.1, time step = 0.05
Energy decrease due to damping

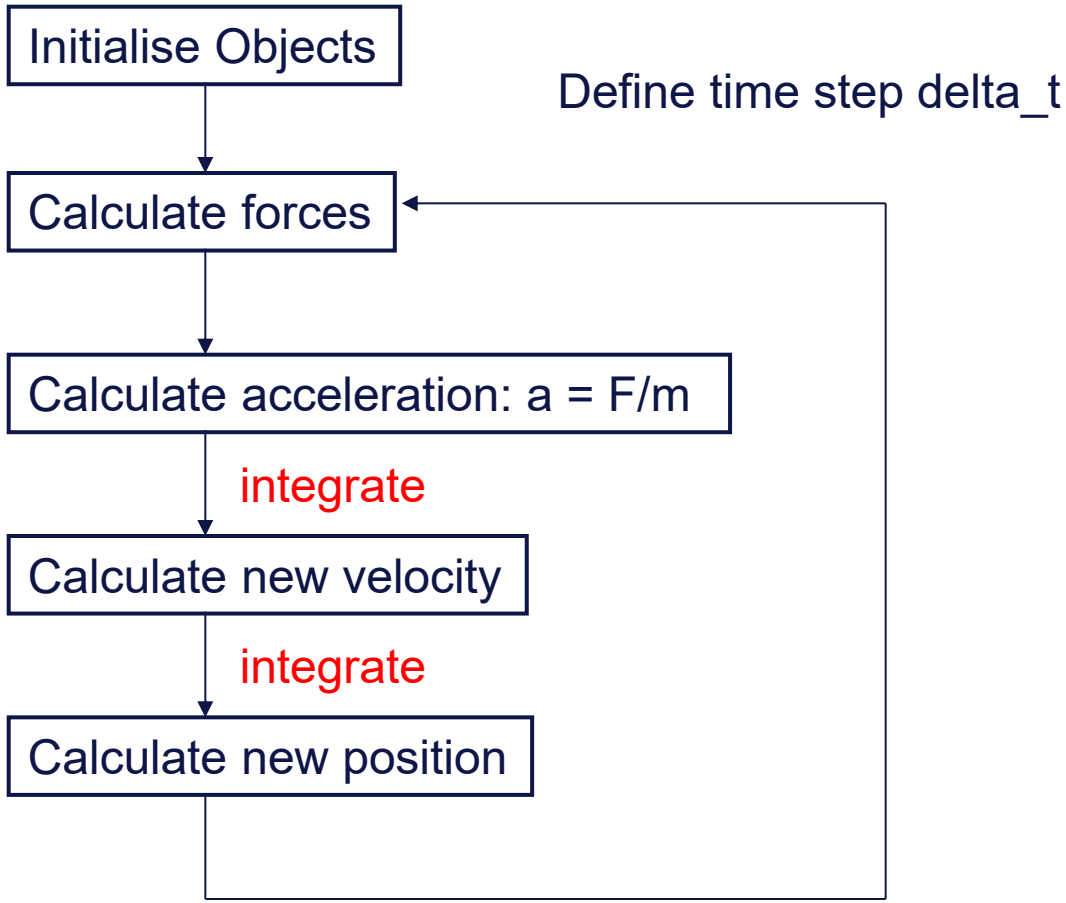
1-dimensional simulation

- Rope
  - A simple simulation of rope can be achieved by connecting a number of spring/dashpot elements together



The point p_0 is constrained eg it is attached to the wall

1-dimensional simulation



Note: No collision detection/response included at the moment

1-dimensional simulation

- Calculating forces
  - Considering mass m_i, $0<i<5$
  - The total force ${}^{T}F_i$ and acceleration ${}^{T}a_i$ on m_i are now:

$$\mathbf{g} = (0,0,-9.81)$$

$${}^{T}\mathbf{F}_i = \mathbf{F}_i - \mathbf{F}_{i-1} + m_i\mathbf{g}$$

$${}^{T}\mathbf{a}_i = \frac{{}^{T}\mathbf{F}_i}{m_i}$$

1-dimensional simulation

- Calculating forces
  - Since m_0 is constrained, the total force acting on it is zero – hence no acceleration
  - m_5 is connected to only one spring and hence the force and acceleration is:

$$^{T}\mathbf{F}_5 = -\mathbf{F}_4 + \mathbf{g}$$

$$^{T}\mathbf{a}_5 = \frac{^{T}\mathbf{F}_5}{m_5}$$

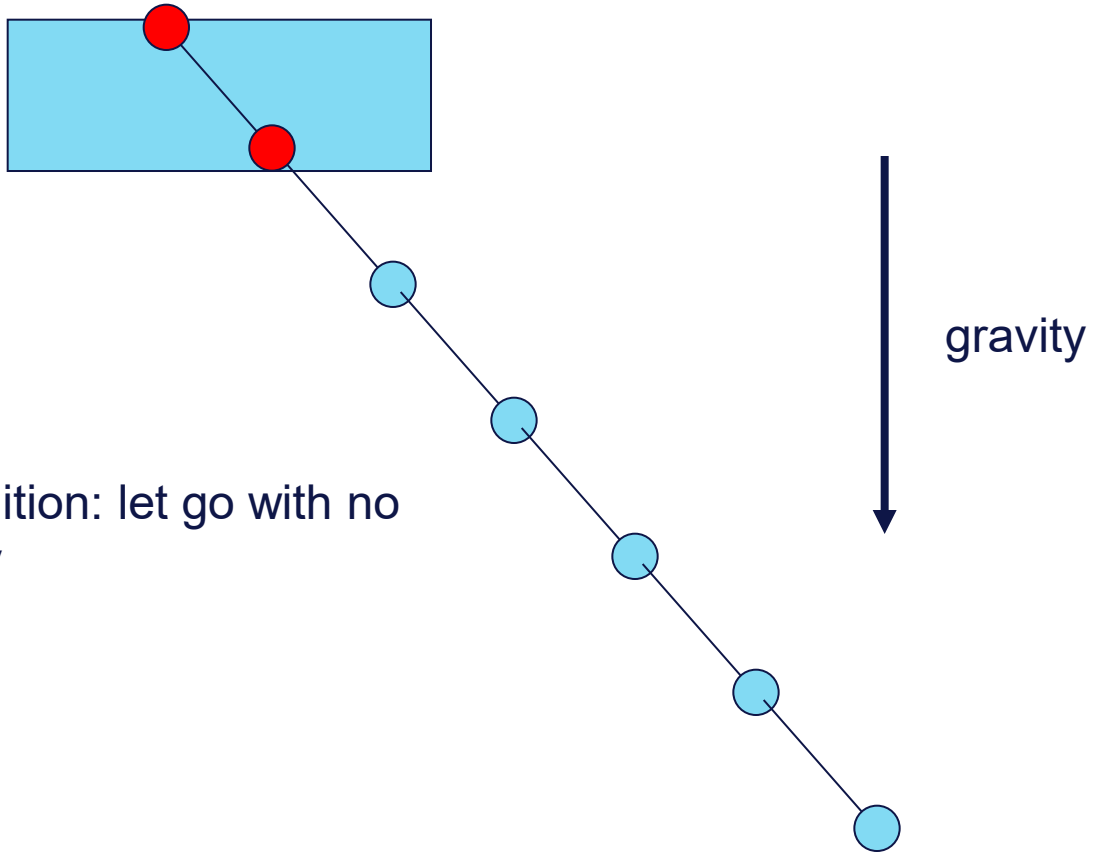
1-dimensional simulation

- Method
  - Collect all forces on all masses
  - Calculate all accelerations on all masses
  - Integrate to produce all velocities
  - Integrate to produce all new positions of the masses
- Note:
  - The current velocity is used in the damping equation
  - No forces act to stop rotations of the springs around a mass point
  - Rope has no rigidity … problem!

1-dimensional simulation

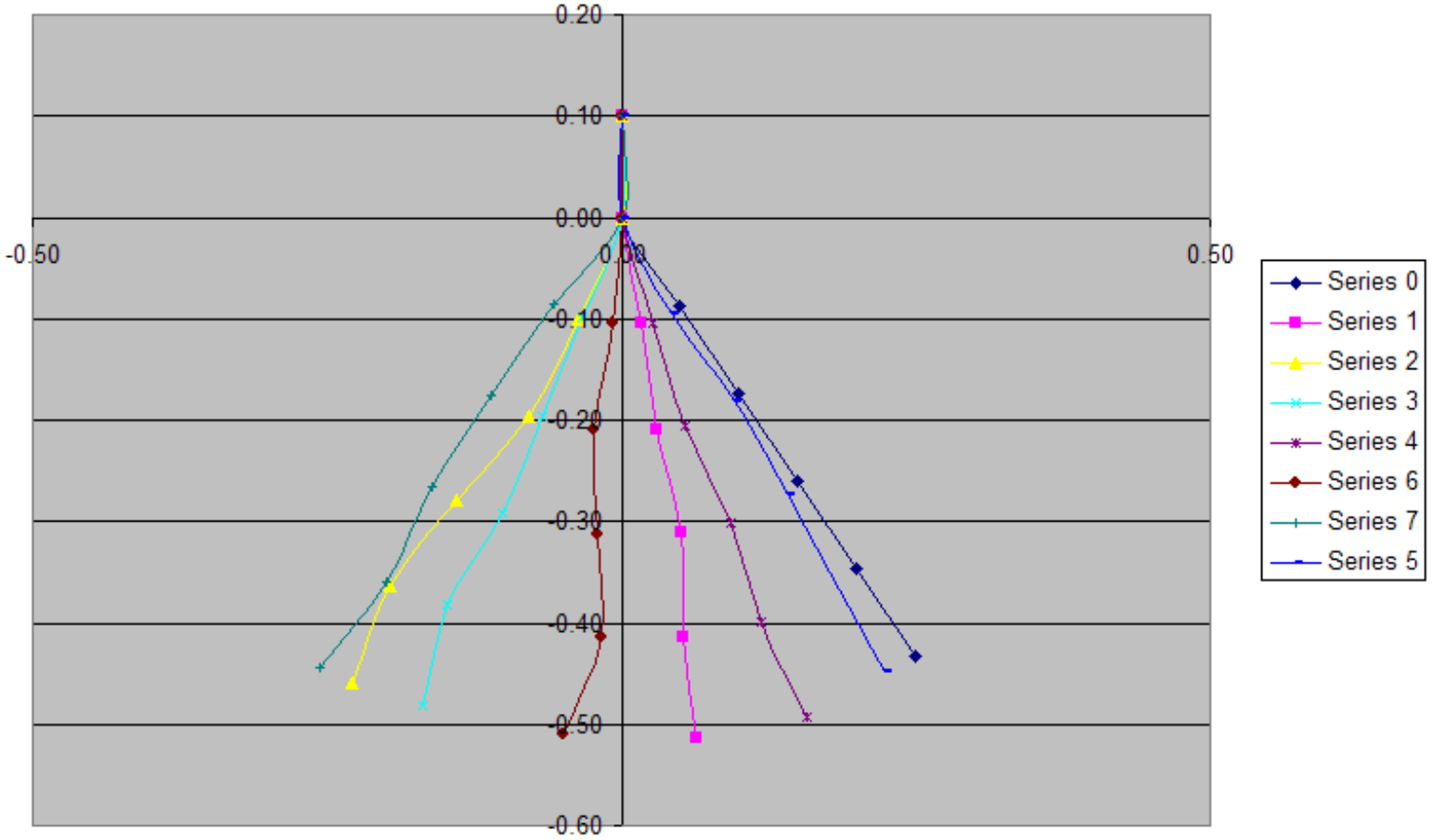
- Example: Rope hanging from pivot
  - Six segments make up rope
  - Top segment is static (eg nodes 0 and 1 fixed)
  - Length of each segment is 0.1m
  - No damping
  - Variable spring coefficient

1-dimensional simulation



Starting condition: let go with no initial velocity

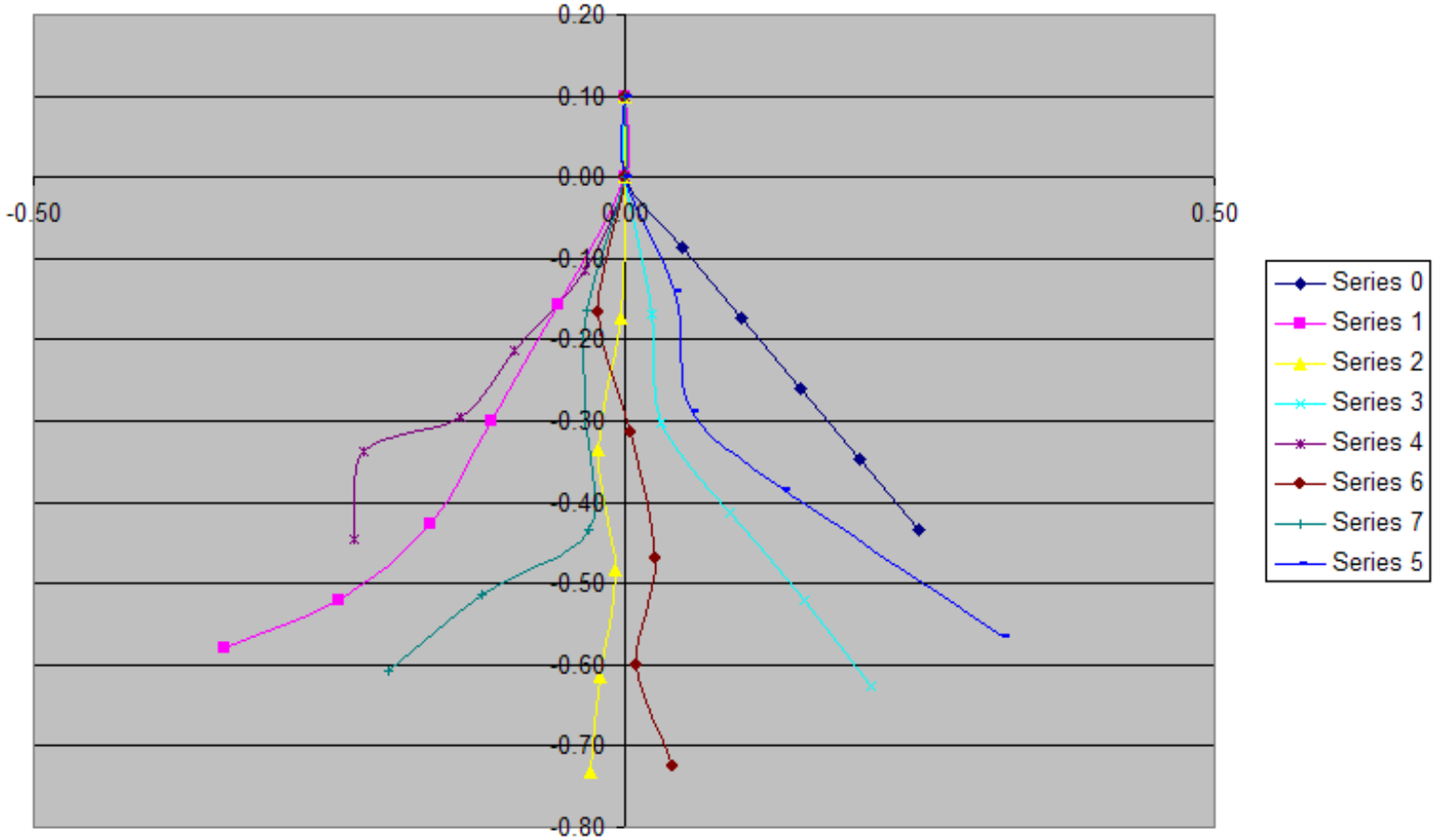
1-dimensional simulation



High spring coefficient (= 1000) and time step = 0.01

Little stretching observed

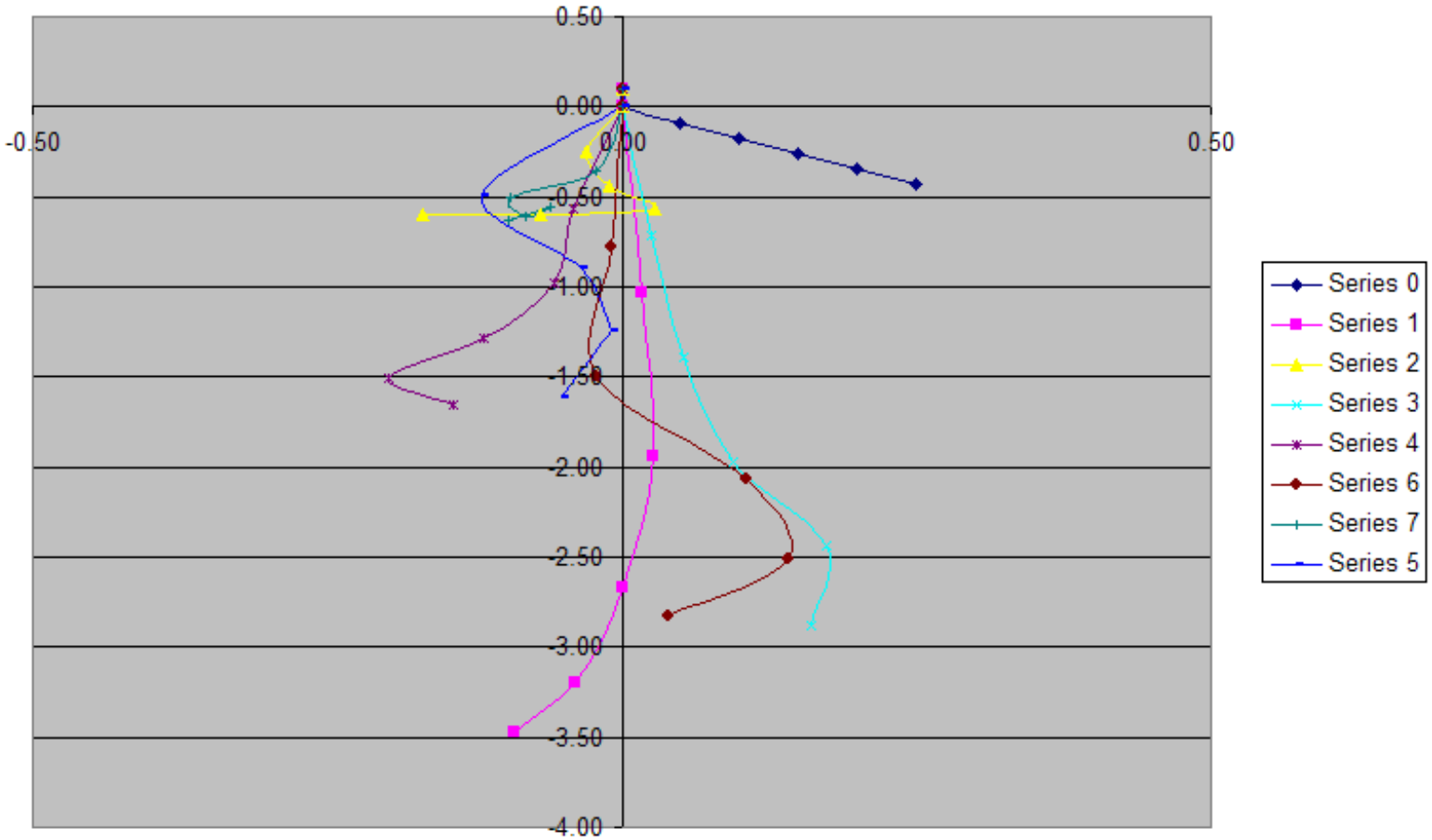
1-dimensional simulation



Medium spring coefficient (= 100) and time step = 0.01

More stretching and deformation observed

1-dimensional simulation

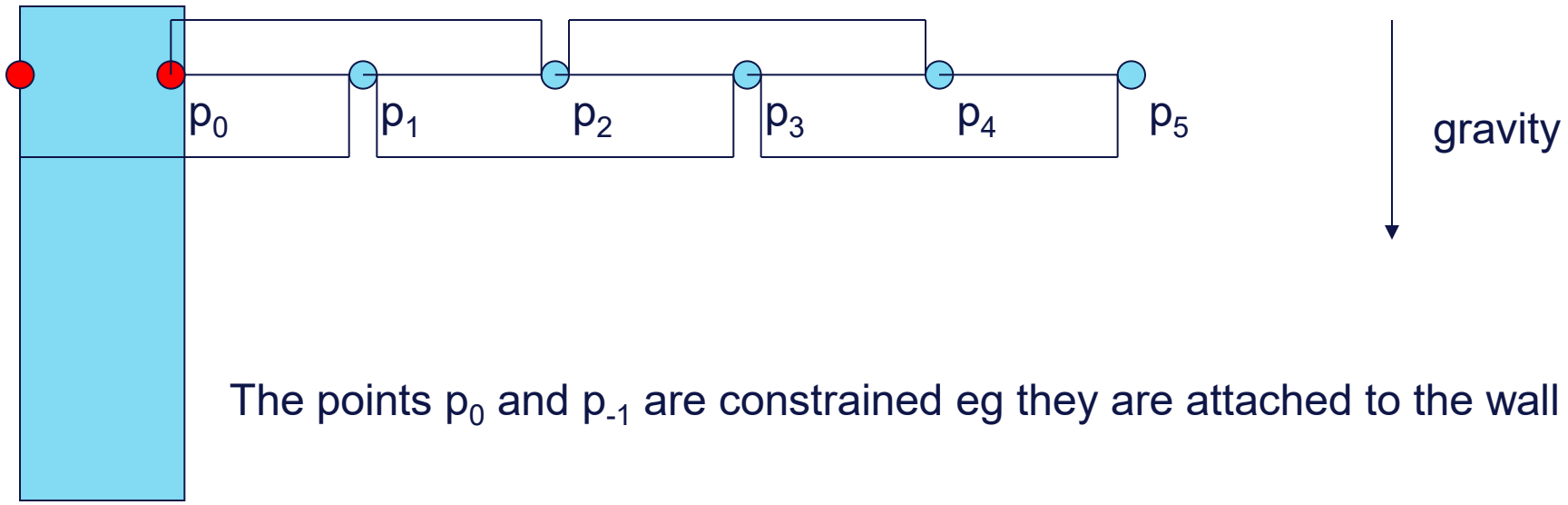


Low spring coefficient (= 10) and time step = 0.01

Large stretching and erratic behaviour

1-dimensional simulation

- Adding flexion springs
  - These extra springs handle the stresses from the cloth bending
  - Flexion springs connect every other mass particle



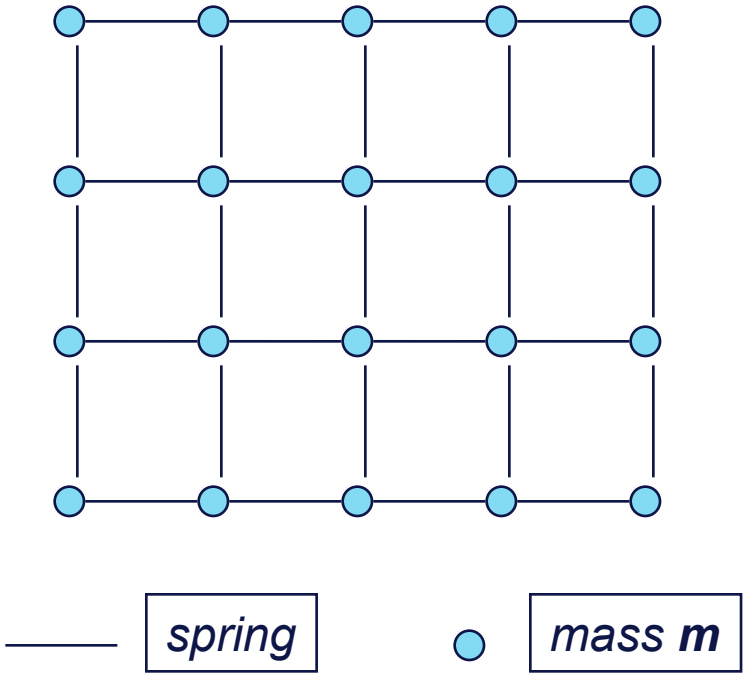
The points p_0 and p_{-1} are constrained eg they are attached to the wall

1-dimensional simulation

- Change to calculation
  - Now need to add they flexion forces onto each mass – two extra forces based upon Hook's law
  - Flexion springs may have a different value of k_s and k_d, depending upon the required results
  - Note:
    - The extra spring included for p_1 may sometimes be ignored – it is an example of a boundary condition problem
    - The greater the number of springs connected to a mass, the greater the chance of numerical inaccuracies
    - The current values of k_s and k_d include aspects of the spring length. If this is not the case then the rest length L_r of the spring should be used to divide the extension of the spring

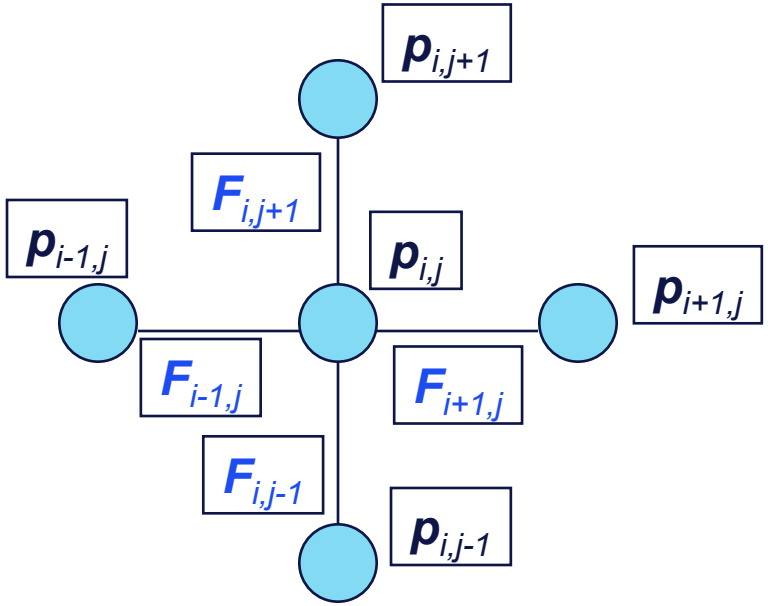
2-dimensional simulation

- Theory
  - Very similar to 1-dimension case
  - Simple example



2-dimensional simulation

- Calculating forces

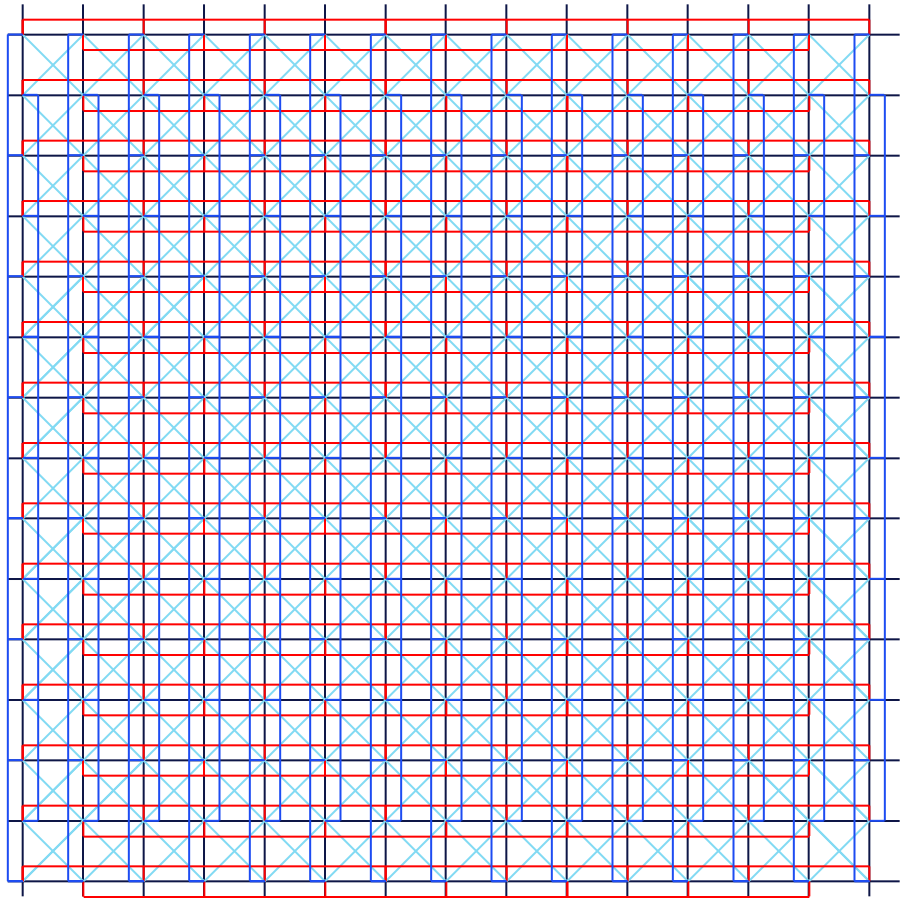


Need to also include gravity and wind if required

2-dimensional simulation

- Calculating forces
  - Add all four forces from the four spring and also gravity as before
  - Need to take care at the boundary of the cloth, edge points only have three forces, corners two
- Problems
  - Bends too easy, needs flexion springs (every other mass connected in horizontal and vertical direction)
  - No simulation of shear stress – add shear springs (diagonal springs)
  - Even more problems with boundary conditions and numerical errors

2-dimensional simulation



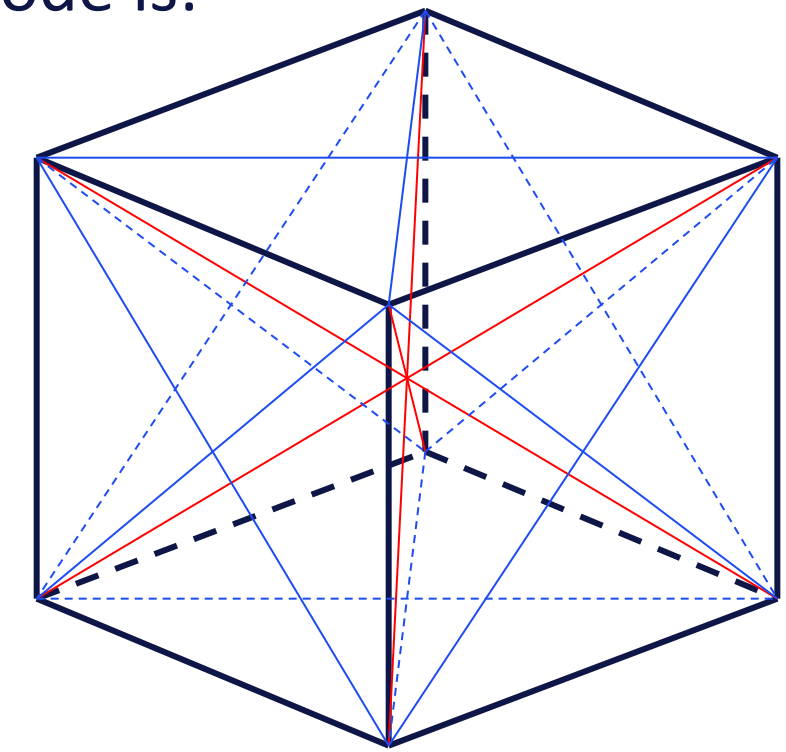
Typical interconnect of springs in a full simulation

2-dimensional simulation

- Connections at a single mass
  - Structural springs: 4
  - Flexion springs: 4
  - Shear springs: 4
  - Total: 12 springs
  - Also need to add in gravity, wind and any forces from collisions

3-dimensional simulation

- Extends 1- and 2-dimensional case
- Simple node is:

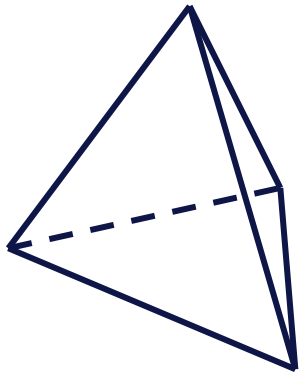


3-dimensional simulation

- Creating an object
  - Mesh object from basic nodes, keeping as regular as possible
  - Allowed to distort geometry but keep topology
  - Equivalent of meshing the surface of an object with triangles: can be difficult to achieve
  - Note:
    - Even more springs!

3-dimensional simulation

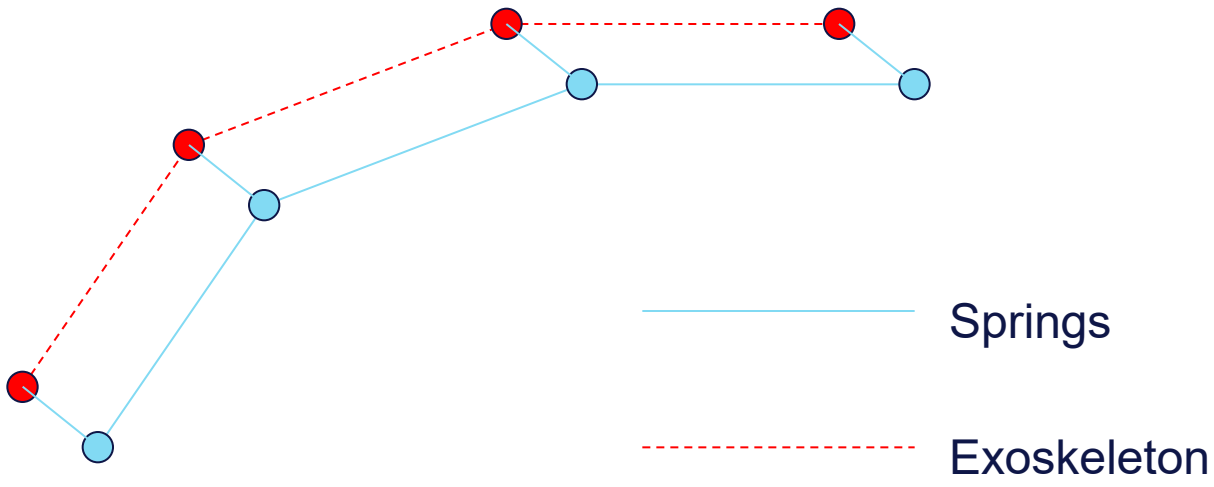
- Alternatives
  - Use tetrahedral elements



- Use a surface representation
  - Include an exoskeleton, an invisible frame which is the same shape as the object
  - Masses are connected to each other and their associated points on the exoskeleton

3-dimensional simulation

- Exoskeleton



Cutting and Tearing

- Application
  - Cutting cloth
  - Incisions in medical simulation
- Methods
  - Dissect elements (eg triangles or squares) and create new elements
    - Difficult, especially insertion of new springs whilst keeping equilibrium
  - Cut between existing elements
    - Easier but requires small initial elements
    - This approach will be described next

Cutting and Tearing

- Assumptions
  - Only consider 2-dimensional case (easily extended to 1- and 3-dimensions)
  - Ignore the use of flexion springs – reasonably easy to put these back but complicates the description
  - Break the object (cloth or flesh) into rectangular plates (elements)
  - Connect plates together with filaments (infinitesimal in length, zero mass)
  - Mass divided into four and distributed to neighbouring vertices on each adjacent plate

Cutting and Tearing

- Representation

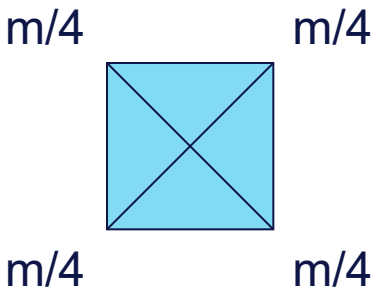
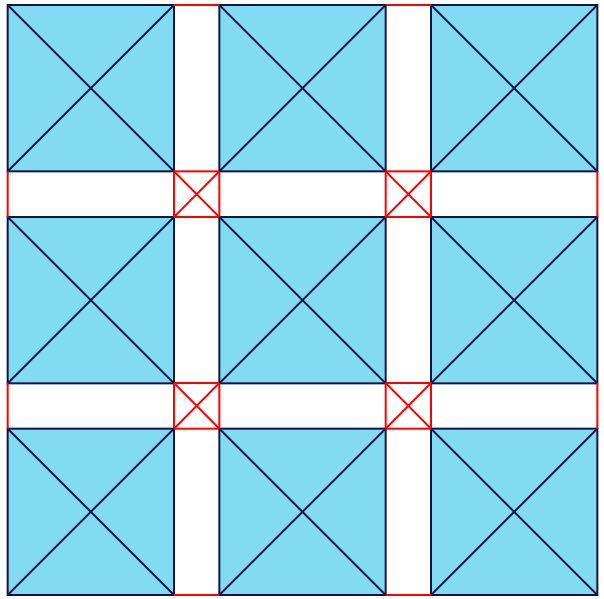


Plate:
Includes structural
and shear springs

Cloth/skin made from plates
connected together with filaments

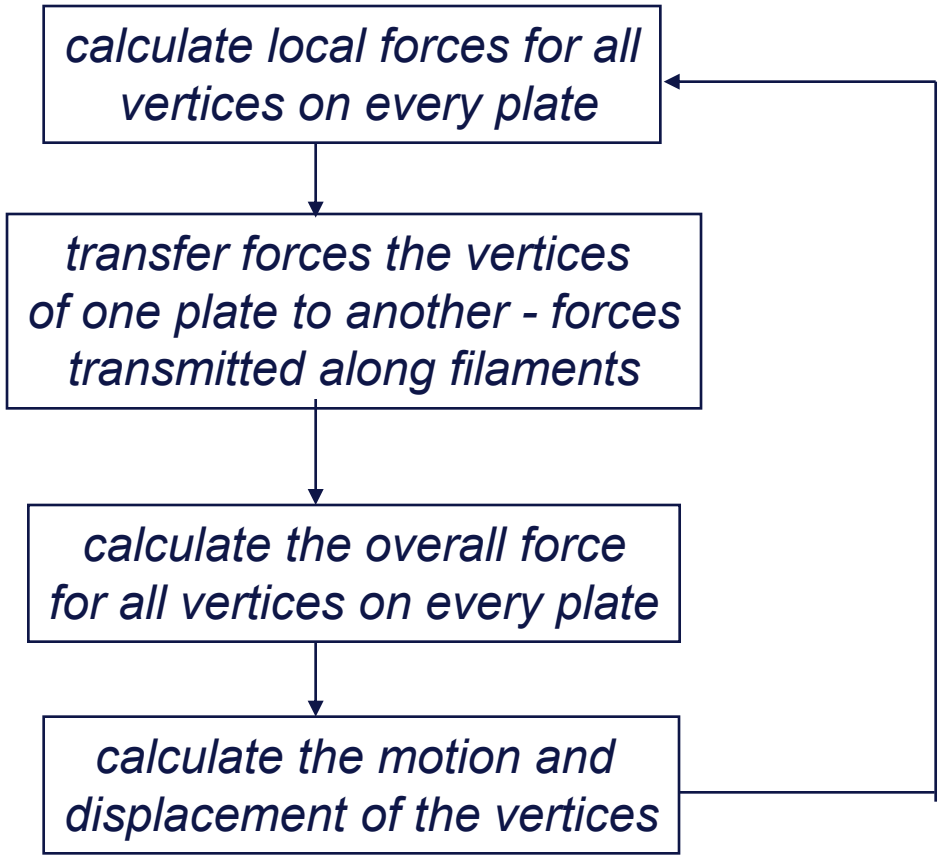


filaments

springs

Cutting and Tearing

- Method



Cutting

- Method
  - Forces are transmitted between plates through filaments
  - Cutting is performed by snapping the filaments between plates, hence stopping forces from being transmitted
  - Whenever a filament is cut, the associated mass vertices are likely to move apart

Tearing

- Method
  - Defining snapping coefficient with each filament. Ideally these will randomly defined but with an overall binomial/normal distribution
  - If the force along a filament exceed the snapping coefficient then the filament snaps
  - By using a random distribution for the snapping coefficient, jagged tears will appear

Mathematical Notes

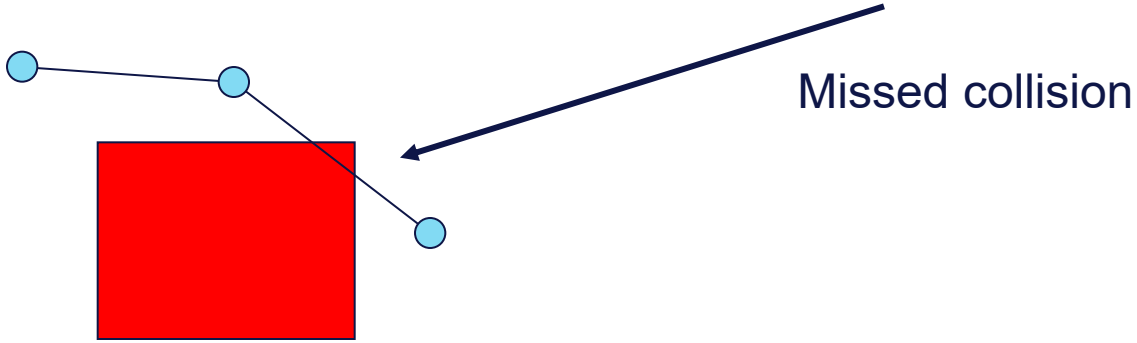
- Energy
  - We have previously considered kinetic and potential energy of an object moving under gravity
  - Now consider the potential energy that exists within a spring
  - Energy in the spring is:

$$E_p = \frac{1}{2} k_s \left(L - L_r\right)^2$$

Note: differentiating this equation gives the force equation for the spring

Collision Detection

- Typical method
  - Cloth vertex with object polygons
    - Quick but has problems when cloth between vertices intersects the object
    - Often claim collision when vertex near object polygon



Collision Detection

- Further conditions
  - Object vertex with cloth polygon
  - Object polygon against cloth polygon
  - Cloth self-collision
- Problems
  - Each of these is more complex
  - Processing time increase dramatically
  - However, cloth self-collision essential for realism

Time to “Code”

Canvas Workshop 6.1 Making a spring force